Agents, Tool Use, and Agentic Frameworks in Production

Scaler School of Technology | ML System Design and LLMOps | Week 4 — Session 2/2 | Pre-Lecture Prep

What Is This Lecture About?

This session builds directly on the RAG system you designed in Week 4, Session 1. You now know how to retrieve relevant context and inject it into an LLM prompt. Today, we ask: what happens when the LLM itself gets to decide what to retrieve, when to retrieve it, which tools to call, and in what order? That is an agent. This session covers what agents are, when to use them, how they break in production, and what frameworks and safeguards exist to make them reliable.

What Makes a System an Agent?

The distinction between a fixed pipeline and an agentic system. Three capabilities that make a model agentic: tool use, multi-step reasoning, and memory. When NOT to use agents — the underrated insight.

Agentic Architectures and Memory

ReAct (Thought + Action + Observation loop), Plan-and-Execute, multi-agent systems with orchestrators and specialists, and the four memory types agents rely on.

Tool Use and Production Failure Modes

How function calling works end-to-end. The six failure modes specific to agentic systems: infinite loops, cost blow-up, context overflow, tool failure cascades, prompt injection, and hallucinated tool calls.

Frameworks and Human-in-the-Loop

Honest comparison of LangChain, LlamaIndex, and plain Python for production. Risk tiering for autonomous actions. The interrupt-and-resume pattern for human oversight.

What to Expect in the Session

- Opening discussion: pipeline or agent? Identify the difference from a description
- ReAct trace walkthrough: reading a live thought-action-observation sequence
- The 6 production failure modes table with concrete examples
- Two short quizzes on tool design and failure mode identification
- Langfuse observability: what a traced agent session looks like
- Human-in-the-loop design: the interrupt-and-resume flow
- Framework comparison: honest production assessment

Prerequisites and Context

This session assumes you have completed Week 4, Session 1 (RAG: Retrieval-Augmented Generation). You should be comfortable with the following concepts:

- Embedding vectors and vector similarity search
- The RAG pipeline: embed query, retrieve chunks, inject into prompt, generate
- LLM context windows and their limits
- Basic prompt engineering: system prompts, few-shot examples
- JSON as a data interchange format

How to Prepare

- **Think about this before the session:** recall a task from your last internship or project that involved automation — a script, a workflow, a batch job. Now imagine an LLM agent with tool access doing that task autonomously. Write down three things that could go wrong if the agent had no safeguards. We will discuss this at the start of class.
- Optional: read the ReAct paper abstract (Yao et al. 2022 — search "ReAct Synergizing Reasoning and Acting Language Models"). One page, freely available.
- Optional: skim the OWASP Top 10 for LLM Applications (owasp.org/www-project-top-10-for-large-language-model-applications/). Focus on item #1 (Prompt Injection) and #6 (Excessive Agency).
- No new tools required. We will look at Langfuse in the browser — no local setup needed.

Duration: ∼2 hours **Prerequisites:** Week 4 Session 1 (RAG), LLM Foundations **Instructor:** Priyansh Saxena